Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal
Turn a rough idea into a precise, buildable project description.

Instructions
Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

_____

2. Problem you want to solve (2–3 short sentences):

_____

_____

3. Target users (who will actually use this?):

_____

4. Inputs (what data / actions will the program receive?):
   - _____
   - _____
5. Outputs (what will the user see / get?):
   - _____
   - _____
6. Constraints or assumptions (keep it realistic):
   - _____
   - _____

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - _____
   - _____
   - _____
2. List optional features (only if time allows):
   - _____
   - _____
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

_____

_____

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
    - Module 1: _____
    - Module 2: _____
    - Module 3: _____
    - Module 4: _____
2. For each module, write its main responsibility in one sentence:
    - Module 1 does: _____
    - Module 2 does: _____
    - Module 3 does: _____
    - Module 4 does: _____

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: _____
2. Functions planned:
    - Function 1 name: _____
        - Input(s): _____
        - Output: _____
    - Function 2 name: _____
        - Input(s): _____
        - Output: _____
    - Function 3 name: _____
        - Input(s): _____
        - Output: _____
3. Draw or briefly describe how data flows between these functions:

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal
Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions
Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

---

2. For this entity, list required fields:
   - Field 1: _____ (type: int/str/float/…)
   - Field 2: _____ (type: _____)
   - Field 3: _____ (type: _____)
   - Field 4: _____ (type: _____)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
   "field1": _________,
   "field2": _________,
   ...
}
```

2. Option B – Class:

python

```
class Item:
   def __init__(self, field1, field2, ...):
      self.field1 = field1
      self.field2 = field2
      ...
```

3. Which option will you use, and why?

---

4. Write a short example of creating and using this structure:

python

```
# Example creation and simple usage
```

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal
Build habit of designing tests before writing code.

Instructions
Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: _____
2. In simple words, what should this function do?

---

3. List at least 5 test cases (inputs and expected outputs):

Test #	**Input(s)**	**Expected output**
1		
2		
3		
4		
5		

4. Now write a small test harness:

python

```
def my_func(...):
    # TODO: implement
    pass


# Tests
print("Test 1:", my_func(...), "expected:", ...)
print("Test 2:", my_func(...), "expected:", ...)
```

5. Reflection:
    - Did writing tests first change how you thought about the function? How?

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal
Identify and protect weak points with validation and exception handling.

Instructions
Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: _____
    - How to detect it? _____
    - How to handle it politely for the user? ____
2. Failure Point 2: _____
    - Detect: _____
    - Handle: _____
3. Failure Point 3: _____
    - Detect: _____
    - Handle: _____

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")


try:
    with open(file_path, "r") as f:
        data = f.read()
    print("File loaded successfully.")
except FileNotFoundError:
    print("File not found. Please check the path.")
except PermissionError:
    print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

```
# Your robust code:
```

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal
Practice cleaning up messy, working code into clear, modular code.

Instructions
Take one working script (from your own work or given by teacher).
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

---

2. Identify at least three problems (long function, repetition, confusing names, etc.):
   - Problem 1: _____
   - Problem 2: _____
   - Problem 3: _____
3. Plan refactoring steps (high level):
   - Step 1: _____
   - Step 2: _____
   - Step 3: _____
4. After refactoring, write or paste your improved version here:

python

 # Refactored code:

5. Reflection:
   - What became easier to understand after refactoring?

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal
Learn to take arguments and options from the command line.

Instructions
Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example:
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?

---

3. Sketch CLI arguments:

Argument / Option	**Meaning**

4. Implement with argparse (skeleton):

python

```
import argparse


parser = argparse.ArgumentParser(description="Describe your tool here")

parser.add_argument("--name", type=str, help="Student name")

parser.add_argument("--score", type=int, help="Student score")

args = parser.parse_args()


# Use args.name, args.score here

print("Name:", args.name)

print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal
Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):
    - Milestone 1: _____
    - Milestone 2: _____
    - Milestone 3: _____
    - Milestone 4: _____
    - Milestone 5: _____
2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):
    - Risk 1: _____
        - Plan B if this fails: _____
    - Risk 2: _____
        - Plan B if this fails: _____
    - Risk 3: _____
        - Plan B if this fails: _____

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

---

2. One thing I now believe about myself as a problem-solver:

---

3. If I start a new project tomorrow, what will I do differently from day 1?